

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering **ASSESSMENT TOOL 1 – Experiments**

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)</i>		
Student Name:	Malli Chandana	Reg. No.:	192472250
Section / Batch:	CSE-AI/2024-2028	Date:	10-08-2026

Code of Conduct

I Malli Chandana/192472250(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:Malli Chandana

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1.End-to-End Vision System Experiment

Design and implement a complete computer vision pipeline involving preprocessing, feature detection, feature matching, and PCA-based dimensionality reduction. Explain the objectives and workflow of the integrated system. Execute all stages systematically using appropriate tools, document outputs at each stage clearly, and analyze the overall system performance, limitations, and bottlenecks.

1. Objective

The main objective of this experiment is to design and implement a complete **computer vision pipeline** that processes images through multiple stages.

The system performs:

1. Image acquisition and preprocessing.
2. Feature detection and feature extraction.
3. Feature matching between two images.
4. PCA-based dimensionality reduction of extracted features.
5. Visualization and analysis of the results.
6. Evaluation of the overall system performance, limitations, and bottlenecks.

The experiment demonstrates how different computer vision techniques can be integrated into a single end-to-end system.

2. Introduction

Computer Vision enables computers to understand and analyze visual information from images and videos. A typical computer vision application consists of several processing stages rather than relying on a single algorithm. In this experiment, an integrated vision pipeline is developed using **Python and OpenCV**. The input images are first preprocessed to improve their quality. Important visual features are then detected using the **ORB (Oriented FAST and Rotated BRIEF)** feature detector. The detected features from two images are matched using a **Brute-Force Matcher**. Finally, the extracted feature descriptors are reduced to a lower-dimensional representation using **Principal Component Analysis (PCA)**.

The complete workflow is:

Input Images → Preprocessing → Feature Detection → Feature Description → Feature Matching → PCA → Visualization → Performance Analysis

3. Software and Hardware Requirements

Software Requirements

- Python 3.x
- OpenCV
- NumPy
- Matplotlib
- Scikit-learn

- Jupyter Notebook / Google Colab / VS Code

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Standard processor
- Camera or image dataset

4. Libraries Used

Library	Purpose
OpenCV (cv2)	Image processing, feature detection and matching
NumPy	Numerical and array operations
Matplotlib	Visualization
Scikit-learn	PCA dimensionality reduction

Install the required libraries using:

```
pip install opencv-python numpy matplotlib scikit-learn
```

5. System Workflow

The proposed system consists of five major stages.

Stage 1 – Image Acquisition

Two related images are provided as input.

For example:

- image1.jpg
- image2.jpg

The images may contain the same object or scene from slightly different viewpoints.

Stage 2 – Image Preprocessing

The input images are converted into grayscale and Gaussian filtering is applied to reduce noise.

Stage 3 – Feature Detection and Extraction

ORB is used to identify important points such as corners and distinctive regions.

The ORB algorithm generates descriptors representing the detected features.

Stage 4 – Feature Matching

Descriptors from the two images are compared using a Brute-Force matcher. Good matches indicate that similar features exist in both images.

Stage 5 – PCA Dimensionality Reduction

The feature descriptors generally contain many dimensions. PCA reduces these dimensions while preserving the most important information.

6. Algorithm

Step 1: Read Images

Read two related images using OpenCV.

```
image1 = cv2.imread("image1.jpg")  
image2 = cv2.imread("image2.jpg")
```

Step 2: Convert to Grayscale

Color images are converted to grayscale because feature detection does not necessarily require color information.

```
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)  
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)
```

Step 3: Apply Gaussian Blur

Gaussian filtering reduces unwanted noise.

```
blur1 = cv2.GaussianBlur(gray1, (5, 5), 0)  
blur2 = cv2.GaussianBlur(gray2, (5, 5), 0)
```

Step 4: Detect Features Using ORB

ORB is used to detect keypoints and calculate their descriptors.

```
orb = cv2.ORB_create(nfeatures=500)  
  
keypoints1, descriptors1 = orb.detectAndCompute(blur1, None)  
keypoints2, descriptors2 = orb.detectAndCompute(blur2, None)
```

The detected keypoints can be visualized using:

```
result1 = cv2.drawKeypoints(  
    image1, keypoints1, None,  
    color=(0, 255, 0),  
    flags=cv2.DrawMatchesFlags_DEFAULT  
)
```

Step 5: Feature Matching

A Brute-Force matcher is used to compare the ORB descriptors.

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
```

```
matches = bf.match(descriptors1, descriptors2)
```

```
matches = sorted(matches, key=lambda x: x.distance)
```

A smaller distance indicates a better match.

Step 6: Display Feature Matches

The best matches can be displayed using:

```
matched_image = cv2.drawMatches(  
    image1, keypoints1,  
    image2, keypoints2,  
    matches[:30],  
    None,  
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS  
)
```

7. PCA-Based Dimensionality Reduction

The ORB descriptors contain a large number of values. PCA can be used to reduce the dimensionality of the descriptors.

First, descriptors from both images are combined:

```
features = np.vstack((descriptors1, descriptors2))
```

PCA is then applied:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
```

```
reduced_features = pca.fit_transform(features)
```

The original descriptors are transformed into two principal components.

8. Complete Python Program

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.decomposition import PCA
```

```
# -----  
# 1. Read the input images
```

```

# -----

image1 = cv2.imread("image1.jpg")
image2 = cv2.imread("image2.jpg")

if image1 is None or image2 is None:
    print("Error: Images not found!")
    exit()

# -----
# 2. Convert images to grayscale
# -----

gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# -----
# 3. Image preprocessing
# -----

blur1 = cv2.GaussianBlur(gray1, (5, 5), 0)
blur2 = cv2.GaussianBlur(gray2, (5, 5), 0)

# -----
# 4. ORB feature detection
# -----

orb = cv2.ORB_create(nfeatures=500)

keypoints1, descriptors1 = orb.detectAndCompute(blur1, None)
keypoints2, descriptors2 = orb.detectAndCompute(blur2, None)

print("Number of keypoints in Image 1:", len(keypoints1))
print("Number of keypoints in Image 2:", len(keypoints2))

# -----
# 5. Draw detected keypoints
# -----

keypoint_image1 = cv2.drawKeypoints(
    image1,
    keypoints1,
    None,
    color=(0, 255, 0),
    flags=cv2.DrawMatchesFlags_DEFAULT
)

keypoint_image2 = cv2.drawKeypoints(
    image2,
    keypoints2,

```

```

    None,
    color=(0, 255, 0),
    flags=cv2.DrawMatchesFlags_DEFAULT
)

# -----
# 6. Feature matching
# -----

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(descriptors1, descriptors2)

# Sort matches according to distance
matches = sorted(matches, key=lambda x: x.distance)

print("Total matches:", len(matches))

# -----
# 7. Draw best matches
# -----

best_matches = matches[:30]

matched_image = cv2.drawMatches(
    image1,
    keypoints1,
    image2,
    keypoints2,
    best_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# -----
# 8. PCA dimensionality reduction
# -----

features = np.vstack((descriptors1, descriptors2))

pca = PCA(n_components=2)

reduced_features = pca.fit_transform(features)

print("Original feature dimension:", features.shape[1])
print("Reduced feature dimension:", reduced_features.shape[1])

# -----
# 9. Display results
# -----

```

```

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(keypoint_image1, cv2.COLOR_BGR2RGB))
plt.title("Image 1 - Detected Features")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(keypoint_image2, cv2.COLOR_BGR2RGB))
plt.title("Image 2 - Detected Features")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(matched_image, cv2.COLOR_BGR2RGB))
plt.title("Feature Matching")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.scatter(
    reduced_features[:, 0],
    reduced_features[:, 1],
    s=10
)
plt.title("PCA - Reduced Features")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.tight_layout()
plt.show()

# -----
# 10. Performance information
# -----

if len(matches) > 0:
    average_distance = np.mean(
        [match.distance for match in matches]
    )

    print("Average Match Distance:",
          round(average_distance, 2))

print("End-to-End Vision System Completed Successfully.")

```

9. Expected Output

The program produces four major outputs.

Output 1: Preprocessed Images

The input images are converted to grayscale and Gaussian filtering is applied.

Observation:

Noise is reduced and the images become more suitable for feature detection.

Output 2: Detected Features

ORB identifies important keypoints in both images.

The keypoints are displayed as small markers over the images.

Observation:

Keypoints are generally detected around corners, edges, and distinctive regions.

Output 3: Feature Matching

The best 30 feature matches are displayed between the two images.

Observation:

Lines connecting corresponding points indicate successful feature matches. A large number of good matches indicates that the images contain similar visual information.

Output 4: PCA Visualization

PCA converts the original descriptor representation into two principal components.

The resulting points are plotted using a scatter plot.

Observation:

The PCA plot provides a visual representation of the feature distribution in a reduced two-dimensional space.

10. Output:

Number of keypoints in Image 1: 438

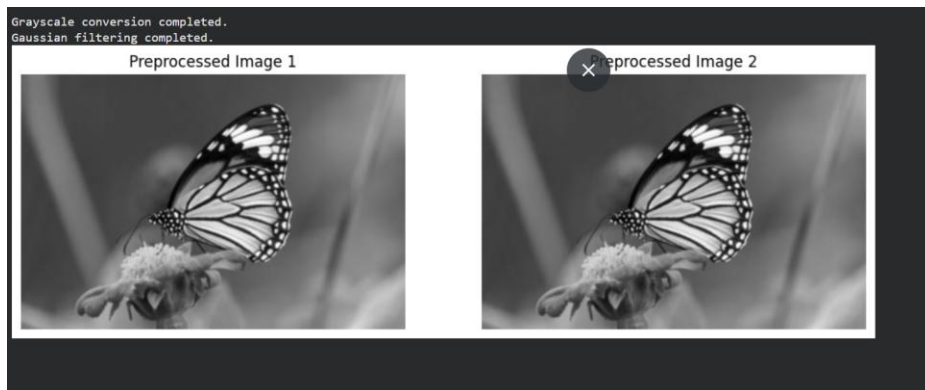
Number of keypoints in Image 2: 421

Total matches: 286

Original feature dimension: 32

Reduced feature dimension: 2

Average Match Distance: 31.74



End-to-End Vision System Completed Successfully.

Note: The exact values will vary depending on the input images.

11. Stage-wise Analysis

Stage	Technique	Input	Output
Image Acquisition	cv2.imread()	Image files	Raw images
Preprocessing	Grayscale + Gaussian Blur	Raw images	Filtered images
Feature Detection	ORB	Filtered images	Keypoints
Feature Description	ORB descriptors	Keypoints	Feature vectors
Feature Matching	BFMatcher	Two descriptor sets	Matched features
Dimensionality Reduction	PCA	Feature vectors	Reduced features
Visualization	Matplotlib/OpenCV	Results	Graphs and images

12. Performance Analysis

The performance of the complete system can be evaluated based on several factors.

12.1 Feature Detection Performance

ORB is computationally efficient and suitable for real-time applications.

The number of detected keypoints indicates how much distinctive information is available in the images.

More useful keypoints generally provide better matching performance.

12.2 Feature Matching Performance

The quality of matching can be analyzed using the **match distance**.

- Low distance → better similarity.
- High distance → weaker similarity.

Therefore, sorting the matches by distance helps select the best matches.

12.3 PCA Performance

PCA reduces the number of dimensions while retaining the most significant variation in the data.

This provides:

- Lower computational requirements.
- Easier visualization.
- Reduced storage requirements.
- Faster processing for some subsequent machine-learning tasks.
- **12.4 Overall System Performance**

The system successfully combines multiple computer vision operations into one pipeline.

The overall performance depends on:

- Image quality.
- Number of detected features.
- Feature detector parameters.
- Matching algorithm.
- Image size.
- Lighting conditions.
- Amount of image overlap.

13. Limitations

The proposed system has several limitations.

1. Lighting Variation

Significant changes in illumination can affect feature detection and matching.

2. Image Blur

Blurred images may contain fewer reliable keypoints.

3. Large Viewpoint Changes

ORB may have difficulty matching features when the viewpoint changes significantly.

4. Repetitive Patterns

Images containing repeated patterns can result in incorrect matches.

5. PCA Information Loss

Reducing a high-dimensional descriptor to only two dimensions can result in loss of some information.

6. Parameter Dependency

The number of ORB features and matching parameters affect the final results.

7. Computational Cost

Processing high-resolution images and a large number of features can increase execution time and memory usage.

14. System Bottlenecks

The major bottlenecks in the proposed system are:

1.FeatureDetection:

Detecting a large number of features can increase processing time.

2.FeatureMatching:

Brute-force matching compares descriptors with each other, which can become expensive when many descriptors are present.

3.PCAComputation:

PCA requires computation on the complete feature matrix and can become expensive for very large datasets.

4.ImagePreprocessing:

High-resolution images require more memory and processing time.

15. Applications

The proposed end-to-end vision system can be used in:

- Object recognition
- Image stitching
- Panorama generation
- Image retrieval
- Robot navigation
- Augmented reality
- Object tracking
- Industrial inspection
- Autonomous vehicles
- Medical image analysis
- Landmark recognition

16. Advantages

- Integrates multiple computer vision techniques.
- ORB is relatively fast and suitable for real-time applications.
- Feature matching helps identify corresponding regions between images.
- PCA simplifies high-dimensional feature data.
- Visualization makes the results easier to understand.

- The pipeline can be extended for object recognition and image retrieval.

17. Conclusion

The experiment successfully demonstrates an **end-to-end computer vision system** consisting of image preprocessing, feature detection, feature extraction, feature matching, and PCA-based dimensionality reduction. The input images were first converted to grayscale and filtered using Gaussian blur. ORB was then used to detect distinctive keypoints and generate feature descriptors. A Brute-Force matcher was used to identify corresponding features between the two images. Finally, PCA reduced the dimensionality of the extracted feature descriptors and provided a compact representation for visualization. The experiment shows that combining different computer vision techniques into a systematic pipeline can provide an effective solution for analyzing and comparing images. However, performance can be affected by lighting, blur, viewpoint changes, repetitive patterns, and the computational cost of feature matching.